Computational Semantic Literature Analysis Pipeline for Cellulose Nanofiber Research

Version: 1.0.0

Author

Alfredo Enrique Villadiego del Villar

PhD Candidate (2025 IFUdG Doctoral Programme)

LEPAMAP–PRODIS Research Group

Universitat de Girona

Girona, Spain

Email: alfredo.villadiego@udg.edu

Description

This notebook implements the complete computational workflow used for the semantic analysis of scientific literature on cellulose nanofibers (CNFs).

The workflow integrates:

- Zotero Web API
- Natural Language Processing (NLP)
- Sentence Embeddings
- BERTopic
- UMAP
- HDBSCAN
- DistilBERT Sentiment Analysis
- Computational Methodological Rigor Assessment
- Publication-quality visualizations

The notebook accompanies the software repository archived on GitHub and Zenodo.

Repository

GitHub:

https://github.com/aevilladel97/Computational-Semantic-Literature-Analysis-Pipeline-for-Cellulose-Nanofiber-Research

---

License

Citation

If this software contributes to your research, please cite the corresponding Zenodo software release.

Last updated: July 2026

Execution Environment

This notebook has been developed and tested exclusively using **Google Colab**.

The installation cells automatically install all required dependencies.

Users only need to provide:

- Zotero User ID
- Zotero Private API Key

No local installation is required.

Computational Semantic Literature Analysis Pipeline

LEPAMAP–PRODIS Research Group Universitat de Girona

Repository: https://github.com/aevilladel97/Computational-Semantic-Literature-Analysis-Pipeline-for-Cellulose-Nanofiber-Research

---

Reproducibility Information

Python: 3.12

Environment: Google Colab

Tested: July 2026

Main libraries: BERTopic Sentence Transformers Transformers PyTorch UMAP HDBSCAN

Exact dependency versions are listed in requirements.txt

Original Contributions

The following components were developed specifically for this software:

• Automated Zotero API retrieval workflow • Semantic model benchmarking framework • Computational methodological rigor scoring algorithm • Publication-quality visualization pipeline • Export manager for publication-ready outputs

External open-source libraries (e.g., BERTopic, UMAP, HDBSCAN, Sentence Transformers, PyTorch) are used under their respective licenses.

Libraries installation

```
!pip install -q pyzotero bertopic sentence-transformers umap-learn hdbscan \
               transformers torch plotly matplotlib pandas numpy tqdm \
               scikit-learn scipy gensim nltk kaleido
!pip install --upgrade pyzotero
!pip install bertopic
!pip install kaleido
!pip install adjustText
!pip install upsetplot
!pip install plotly==5.24.1 kaleido==0.2.1
```

Mostrar el resultado oculto

Literature review and screening through semantic models benchmarking and UMAP best hyperparameters

```
"""
TEMPO-CNF Literature Analysis Pipeline
================================================================================
Author:      Alfredo E. Villadiego del Villar
Affiliation: Universitat de Girona – Lepamap / prodis
Date:        June 2026
Purpose:     Systematic literature review with NLP (BERTopic, sentiment analysis)
             and advanced topic evaluation (C_v coherence, diversity, silhouette).
             Includes UMAP hyperparameter tuning and domain-model benchmarking.
================================================================================
"""

import os, re, time, warnings, zipfile, itertools
from pathlib import Path
from collections import defaultdict
import pandas as pd
import numpy as np
from tqdm import tqdm
from pyzotero import Zotero
from bertopic import BERTopic
from sentence_transformers import SentenceTransformer
from sklearn.feature_extraction.text import CountVectorizer
```

```
from sklearn.feature_extraction.text import CountVectorizer
from umap import UMAP
from hdbscan import HDBSCAN
from transformers import pipeline, AutoTokenizer, AutoModel
import torch
from adjustText import adjust_text
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.graph_objects as go
import plotly.express as px
from typing import List, Dict, Any, Tuple
import nltk
from nltk.stem import WordNetLemmatizer
from gensim.corpora import Dictionary
from gensim.models import CoherenceModel
from sklearn.metrics import silhouette_score

# Ensure NLTK data is present
nltk.download('wordnet', quiet=True)
nltk.download('omw-1.4', quiet=True)

# ------------------------------------------------------------------------------
# CONFIGURATION
# ------------------------------------------------------------------------------
SEED = 42
np.random.seed(SEED)
import random
random.seed(SEED)

OUTPUT_DIR = Path('TEMPO_Review_Outputs')
OUTPUT_DIR.mkdir(exist_ok=True)

try:
    from google.colab import userdata, files
    IN_COLAB = True
except ImportError:
    IN_COLAB = False

warnings.filterwarnings('ignore')
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'

plt.rcParams['font.family'] = 'sans-serif'
plt.rcParams['font.sans-serif'] = ['Arial', 'DejaVu Sans', 'Helvetica', 'Liberation Sans']
plt.rcParams['font.size'] = 10
plt.rcParams['pdf.fonttype'] = 42
plt.rcParams['ps.fonttype'] = 42

# ------------------------------------------------------------------------------
# UMAP HYPERPARAMETER GRID (for tuning)
# ------------------------------------------------------------------------------
UMAP_PARAM_GRID = {
    'n_neighbors': [15, 20, 25],
    'min_dist': [0.0, 0.05, 0.1],
    'n_components': [5, 7, 10]
}

# ------------------------------------------------------------------------------
# UTILITY FUNCTIONS
# ------------------------------------------------------------------------------
def normalize_string(text: str) -> str:
    if not isinstance(text, str):
        return ""
    import unicodedata
    text = ''.join(c for c in unicodedata.normalize('NFD', text)
                   if unicodedata.category(c) != 'Mn')
    text = text.lower()
    text = re.sub(r'[^a-z0-9\\s]', '', text)
    return re.sub(r'\\s+', ' ', text).strip()

def normalize_doi(doi: str) -> str:
    return doi.lower().strip() if isinstance(doi, str) else ""

def extract_technique_from_paths(paths: List[List[str]]) -> str:
    for path in paths:
        if path and any(folder.strip().lower().endswith(' - tempo') for folder in path):
            return 'TEMPO'
    return 'Control'

```

```
def extract_macro_theme_from_paths(paths: List[List[str]]) -> str:
    for path in paths:
        if path:
            for folder in path:
                if folder.strip().lower().endswith(' - tempo'):
                    return folder[:-8].strip()
            return path[-1].strip()
    return 'Unknown'

# ------------------------------------------------------------------------------
# ZOTERO DATA EXTRACTION
# ------------------------------------------------------------------------------
class ZoteroCollectionFetcher:
    def __init__(self, library_id: str, api_key: str):
        self.zot = Zotero(library_id, 'user', api_key)
        self.key_to_path = {}

    def build_collection_tree(self):
        top_colls = self.zot.collections(limit=100)
        stack = [(c, [], []) for c in top_colls if isinstance(c, dict)]
        visited = set()
        while stack:
            coll, parent, path_so_far = stack.pop()
            if not isinstance(coll, dict):
                continue
            key = coll.get('key')
            if key in visited:
                continue
            visited.add(key)
            name = coll.get('data', {}).get('name', 'Unknown')
            full_path = path_so_far + [name]
            self.key_to_path[key] = full_path
            try:
                subs = self.zot.collections(key, limit=100)
                for sub in subs:
                    if isinstance(sub, dict):
                        stack.append((sub, None, full_path))
            except Exception:
                pass
        return self.key_to_path

    def fetch_all_items(self) -> List[Dict]:
        all_items = []
        start = 0
        limit = 50
        while True:
            try:
                items = self.zot.items(limit=limit, start=start,
                                       params={'include': 'collections'})
                if not items:
                    break
                for item in items:
                    if not isinstance(item, dict):
                        continue
                    data = item.get('data', {})
                    item_key = data.get('key')
                    coll_keys = data.get('collections', [])
                    paths = [self.key_to_path.get(ck, []) for ck in coll_keys if ck in self.key_to_path]
                    enriched = {
                        'key': item_key,
                        'title': data.get('title', ''),
                        'doi': data.get('DOI', ''),
                        'abstract': data.get('abstractNote', ''),
                        'publication_year': data.get('date', ''),
                        'authors': data.get('creators', []),
                        'item_type': data.get('itemType', ''),
                        'collection_paths': paths,
                        'collection_keys': coll_keys,
                    }
                    all_items.append(enriched)
                start += limit
            except Exception:
                start += limit
                time.sleep(1)
        return all_items
```

```
class ZoteroDataProcessor:
    def __init__(self, items: List[Dict]):
        self.items = items

    def classify_items(self) -> pd.DataFrame:
        records = []
        for item in self.items:
            paths = item.get('collection_paths', [])
            technique = extract_technique_from_paths(paths)
            macro_theme = extract_macro_theme_from_paths(paths)
            records.append({
                'key': item['key'],
                'title': item['title'],
                'doi': item['doi'],
                'abstract': item['abstract'],
                'publication_year': item['publication_year'],
                'authors': item['authors'],
                'item_type': item['item_type'],
                'technique': technique,
                'macro_theme': macro_theme,
                'collection_paths': paths
            })
        return pd.DataFrame(records)

    def deduplicate_items(self, df: pd.DataFrame):
        df_dedup = df.copy()
        df_dedup['doi_norm'] = df_dedup['doi'].apply(normalize_doi)
        df_dedup['title_norm'] = df_dedup['title'].apply(normalize_string)

        mask_doi = df_dedup['doi_norm'] != ''
        dup_doi = df_dedup[mask_doi].duplicated(subset='doi_norm', keep=False)
        for doi, group in df_dedup[mask_doi & dup_doi].groupby('doi_norm'):
            if (group['technique'] == 'TEMPO').any():
                df_dedup = df_dedup.drop(group[group['technique'] != 'TEMPO'].index)

        no_doi = df_dedup['doi_norm'] == ''
        dup_title = df_dedup[no_doi].duplicated(subset='title_norm', keep=False)
        for title, group in df_dedup[no_doi & dup_title].groupby('title_norm'):
            if (group['technique'] == 'TEMPO').any():
                df_dedup = df_dedup.drop(group[group['technique'] != 'TEMPO'].index)

        df_dedup = df_dedup.drop_duplicates(subset='title_norm', keep='first')
        df_dedup = df_dedup.drop(columns=['doi_norm', 'title_norm'])
        return df_dedup

    def filter_by_abstract(self, df: pd.DataFrame) -> pd.DataFrame:
        return df[df['abstract'].notna() & (df['abstract'].str.len() > 0)].copy()

    def extract_publication_year(self, df: pd.DataFrame) -> pd.DataFrame:
        def get_year(x):
            if pd.isna(x) or not isinstance(x, str):
                return None
            m = re.search(r'\b(19|20)\d{2}\b', str(x))
            return int(m.group()) if m else None
        df['publication_year'] = df['publication_year'].apply(get_year)
        return df

    def process_pipeline(self) -> pd.DataFrame:
        df = self.classify_items()
        df = self.deduplicate_items(df)
        df = self.filter_by_abstract(df)
        df = self.extract_publication_year(df)
        return df


# ------------------------------------------------------------------------------
# EMBEDDING MODEL LOADER (supporting both general and domain-specific)
# ------------------------------------------------------------------------------
def load_embedding_model(model_name: str = 'all-mpnet-base-v2'):
    """Load sentence transformer or HuggingFace model for embeddings."""
    if 'matscibert' in model_name.lower() or 'matbert' in model_name.lower():
        tokenizer = AutoTokenizer.from_pretrained(model_name)
        model = AutoModel.from_pretrained(model_name)
        def encode_fn(texts, batch_size=32):
            all_embeddings = []
            for i in range(0, len(texts), batch_size):
                batch = texts[i:i+batch_size]
                encoded = tokenizer(batch, padding=True, truncation=True, return_tensors='pt', max_length=512)
```

```
                with torch.no_grad():
                    outputs = model(**encoded)
                    attention_mask = encoded['attention_mask']
                    embeddings = (outputs.last_hidden_state * attention_mask.unsqueeze(-1)).sum(1) / attention_mask.sum(1)
                all_embeddings.append(embeddings.cpu().numpy())
            return np.vstack(all_embeddings)
        return encode_fn
    else:
        model = SentenceTransformer(model_name)
        return lambda texts, batch_size=32: model.encode(texts, show_progress_bar=False, batch_size=batch_size, convert_to

# ------------------------------------------------------------------------
# TOPIC MODELING PIPELINE
# ------------------------------------------------------------------------
class TopicModelingPipeline:
    def __init__(self, df: pd.DataFrame, seed: int = SEED):
        self.df = df.copy()
        self.seed = seed
        self.model = None
        self.embeddings = None
        self.umap_embeddings = None
        self.coherence_scores = {}
        self.cv_coherence = None
        self.topic_diversity = None
        self.silhouette_highdim = None
        self.silhouette_umap = None

    def prepare_documents(self) -> List[str]:
        return self.df['abstract'].fillna('').apply(lambda x: re.sub(r'\s+', ' ', x).strip()).tolist()

    def compute_embeddings(self, documents: List[str], model_name='all-mpnet-base-v2') -> np.ndarray:
        embed_fn = load_embedding_model(model_name)
        self.embeddings = embed_fn(documents)
        return self.embeddings

    def train_topic_model(self, documents: List[str], embeddings: np.ndarray,
                          umap_kwargs: dict = None):
        if umap_kwargs is None:
            umap_kwargs = {'n_neighbors': 15, 'n_components': 5, 'min_dist': 0.0}
        umap_model = UMAP(metric='cosine', random_state=self.seed, **umap_kwargs)
        hdbscan_model = HDBSCAN(min_cluster_size=12, min_samples=3,
                                cluster_selection_method='eom', prediction_data=True)
        vectorizer_model = CountVectorizer(stop_words='english', ngram_range=(1,2),
                                           max_features=10000, min_df=2, max_df=0.95)
        model = BERTopic(embedding_model=None,
                         umap_model=umap_model,
                         hdbscan_model=hdbscan_model,
                         vectorizer_model=vectorizer_model,
                         language='english',
                         calculate_probabilities=True,
                         nr_topics=20,
                         verbose=False)
        topics, _ = model.fit_transform(documents, embeddings)
        self.model = model
        self.umap_embeddings = self.model.umap_model.transform(embeddings)
        return model

    def handle_outliers(self, documents: List[str]):
        try:
            self.model.reduce_outliers(documents, self.model.topics_,
                                       strategy='c-tf-idf', threshold=0.1)
        except Exception:
            pass

    def compute_metrics(self, documents: List[str]):
        # Simple coherence (co-occurrence of top 5 words)
        topics = set(self.model.topics_) - {-1}
        for tid in topics:
            words = [w for w, _ in self.model.get_topic(tid)[:5]]
            cooc = sum(1 for doc in documents if sum(1 for w in words if w.lower() in doc.lower()) > 1)
            self.coherence_scores[tid] = cooc / len(documents) if documents else 0.0

        # Advanced metrics
        tokenized_docs = [doc.lower().split() for doc in documents]
        dictionary = Dictionary(tokenized_docs)
        topic_words = []
        for tid in set(self.model.topics_) - {-1}:
```

```
            words = [w for w, _ in self.model.get_topic(tid)[:10]]
            topic_words.append(words)
        coherence_model = CoherenceModel(topics=topic_words,
                                         texts=tokenized_docs,
                                         dictionary=dictionary,
                                         coherence='c_v')
        self.cv_coherence = coherence_model.get_coherence()
        all_words = [w for words in topic_words for w in words]
        unique_words = set(all_words)
        self.topic_diversity = len(unique_words) / len(all_words) if all_words else 0.0

        # Silhouette scores
        topics_arr = np.array(self.model.topics_)
        valid_idx = np.where(topics_arr != -1)[0]
        if len(valid_idx) >= 2 and len(np.unique(topics_arr[valid_idx])) >= 2:
            self.silhouette_highdim = silhouette_score(self.embeddings[valid_idx], topics_arr[valid_idx], metric='euclidea
            if self.umap_embeddings is not None:
                umap_2d = self.umap_embeddings[valid_idx, :2]
                self.silhouette_umap = silhouette_score(umap_2d, topics_arr[valid_idx], metric='euclidean')
        else:
            self.silhouette_highdim = np.nan
            self.silhouette_umap = np.nan

    def get_topic_labels(self, n: int = 5) -> Dict[int, str]:
        lemmatizer = WordNetLemmatizer()
        labels = {-1: "Not Grouped (Noise)"}
        for tid in set(self.model.topics_) - {-1}:
            topic_words = self.model.get_topic(tid)
            if not topic_words:
                labels[tid] = f"Topic_{tid}"
                continue
            lemma_weights = defaultdict(float)
            for word, weight in topic_words:
                lemma = lemmatizer.lemmatize(word.lower(), pos='n')
                lemma_weights[lemma] += weight
            sorted_lemmas = sorted(lemma_weights.items(), key=lambda x: x[1], reverse=True)[:n]
            labels[tid] = ' + '.join([lemma for lemma, _ in sorted_lemmas])
        return labels

    def tune_umap_params(self, documents: List[str], embeddings: np.ndarray,
                         param_grid: dict = None) -> dict:
        if param_grid is None:
            param_grid = UMAP_PARAM_GRID
        keys = param_grid.keys()
        best_score = -1
        best_params = None
        results = []
        for values in itertools.product(*param_grid.values()):
            params = dict(zip(keys, values))
            print(f"Testing UMAP params: {params}")
            umap_model = UMAP(metric='cosine', random_state=self.seed, **params)
            reduced = umap_model.fit_transform(embeddings)
            clusterer = HDBSCAN(min_cluster_size=12, min_samples=3, cluster_selection_method='eom')
            topics = clusterer.fit_predict(reduced)
            valid = topics != -1
            if np.sum(valid) > 1 and len(np.unique(topics[valid])) > 1:
                score = silhouette_score(reduced[valid], topics[valid], metric='euclidean')
            else:
                score = -1
            results.append((params, score))
            print(f"  Silhouette score: {score:.4f}")
            if score > best_score:
                best_score = score
                best_params = params
        print(f"Best UMAP params: {best_params} with silhouette {best_score:.4f}")
        self.best_umap_params = best_params
        return best_params

    def train_pipeline(self, model_name='all-mpnet-base-v2', tune_umap=True):
        docs = self.prepare_documents()
        self.compute_embeddings(docs, model_name=model_name)
        if tune_umap:
            best_params = self.tune_umap_params(docs, self.embeddings)
        else:
            best_params = {'n_neighbors': 15, 'n_components': 5, 'min_dist': 0.0}
        self.train_topic_model(docs, self.embeddings, umap_kwargs=best_params)
        self.handle_outliers(docs)
        self.compute_metrics(docs)
```

```
        labels = self.get_topic_labels()
        try:
            self.model._hierarchical_topics = self.model.hierarchical_topics(docs)
        except Exception:
            self.model._hierarchical_topics = None
        return self.model, labels


# ------------------------------------------------------------------------------
# BENCHMARKING: Compare general vs domain-specific models
# ------------------------------------------------------------------------------
def benchmark_models(df, model_names: List[str] = ['all-mpnet-base-v2', 'm3hrdadfi/matbert']):
    results = []
    for model_name in model_names:
        print(f"\n--- Benchmarking model: {model_name} ---")
        pipe = TopicModelingPipeline(df)
        model, labels = pipe.train_pipeline(model_name=model_name, tune_umap=False)
        results.append({
            'model': model_name,
            'C_v_coherence': pipe.cv_coherence,
            'topic_diversity': pipe.topic_diversity,
            'silhouette_highdim': pipe.silhouette_highdim,
            'silhouette_umap': pipe.silhouette_umap,
            'num_topics': len(set(model.topics_) - {-1}),
            'noise_ratio': sum(1 for t in model.topics_ if t == -1) / len(model.topics_)
        })
    benchmark_df = pd.DataFrame(results)
    benchmark_df.to_csv(OUTPUT_DIR / 'Model_Benchmark.csv', index=False)
    print(benchmark_df)
    return benchmark_df

# ------------------------------------------------------------------------------
# SENTIMENT ANALYSIS
# ------------------------------------------------------------------------------
class SentimentAnalysisPipeline:
    def __init__(self, df: pd.DataFrame):
        self.df = df.copy()
        self.model = None

    def load_model(self):
        try:
            self.model = pipeline("text-classification",
                                  model="lxyuan/distilbert-base-multilingual-cased-sentiments-student",
                                  batch_size=16,
                                  device=0 if torch.cuda.is_available() else -1)
        except Exception:
            self.model = None

    def analyze_sentiments(self) -> pd.DataFrame:
        abstracts = self.df['abstract'].fillna('').tolist()
        sentiments = []
        if self.model:
            for i in range(0, len(abstracts), 32):
                batch = [t[:512] for t in abstracts[i:i+32]]
                preds = self.model(batch)
                for p in preds:
                    label = p['label'].lower()
                    if 'pos' in label:
                        sentiments.append('positive')
                    elif 'neg' in label:
                        sentiments.append('negative')
                    else:
                        sentiments.append('neutral')
        else:
            sentiments = ['neutral'] * len(abstracts)
        self.df['sentiment'] = sentiments
        self.df['sentiment_confidence'] = 0.5
        return self.df


# ------------------------------------------------------------------------------
# PUBLICATION-QUALITY VISUALISATIONS
# ------------------------------------------------------------------------------
class PublicationQualityVisualizations:
    def __init__(self, df, model, labels, embeddings=None):
        self.df = df
        self.model = model
```

```
        self.labels = labels
        self.embeddings = embeddings
        self.output_dir = OUTPUT_DIR

    def plot_class_comparison(self, top_n=15):
        top_topics = self.df['topic_id'].value_counts().head(top_n).index.tolist()
        data = []
        for tid in top_topics:
            data.append({
                'topic': self.labels.get(tid, f"T{tid}"),
                'TEMPO': ((self.df['topic_id'] == tid) & (self.df['technique'] == 'TEMPO')).sum(),
                'Control': ((self.df['topic_id'] == tid) & (self.df['technique'] == 'Control')).sum()
            })
        comp = pd.DataFrame(data).sort_values('TEMPO', ascending=True)
        fig, ax = plt.subplots(figsize=(16,10))
        y = np.arange(len(comp))
        width = 0.35
        ax.barh(y - width/2, comp['TEMPO'], width, label='TEMPO',
                color='#4A4A4A', edgecolor='black', hatch='//')
        ax.barh(y + width/2, comp['Control'], width, label='Control',
                color='#A9A9A9', edgecolor='black', hatch='\\\\')
        ax.set_yticks(y)
        ax.set_yticklabels(comp['topic'], fontsize=12)
        ax.set_xlabel('Documents')
        ax.set_title('Topic Distribution: TEMPO vs Control', fontsize=14, fontweight='bold')
        ax.legend()
        plt.tight_layout()
        plt.savefig(self.output_dir/'Class_Comparison.png', dpi=300)
        plt.close()

    def plot_sentiment_by_topic(self, top_n=15):
        top_topics = self.df['topic_id'].value_counts().head(top_n).index.tolist()
        rows = []
        for tid in reversed(top_topics):
            sub = self.df[self.df['topic_id'] == tid]
            rows.append({
                'topic': self.labels.get(tid, f"T{tid}"),
                'positive': (sub['sentiment'] == 'positive').sum(),
                'neutral': (sub['sentiment'] == 'neutral').sum(),
                'negative': (sub['sentiment'] == 'negative').sum()
            })
        sent_df = pd.DataFrame(rows).set_index('topic')
        fig, ax = plt.subplots(figsize=(16,10))
        colors = ['#4A4A4A','#A9A9A9','#2F2F2F']
        hatches = ['//','\\\\','||']
        left = np.zeros(len(sent_df))
        for col, color, hatch in zip(['positive','neutral','negative'], colors, hatches):
            ax.barh(sent_df.index, sent_df[col], left=left, color=color,
                    edgecolor='black', hatch=hatch, label=col.capitalize())
            left += sent_df[col].values
        ax.set_xlabel('Number of Documents', fontsize=14, fontweight='bold')
        ax.set_title('Sentiment Distribution per Topic', fontsize=14, fontweight='bold')
        ax.tick_params(axis='y', labelsize=14)
        ax.legend()
        plt.tight_layout()
        plt.savefig(self.output_dir/'Sentiment_by_Topic.png', dpi=300)
        plt.close()

    def plot_sentiment_by_class(self):
        cross = pd.crosstab(self.df['technique'], self.df['sentiment'])
        for cls in ['TEMPO','Control']:
            if cls not in cross.index:
                cross.loc[cls] = 0
        cross = cross.reindex(['TEMPO','Control'])
        fig, ax = plt.subplots(figsize=(10,6))
        colors = ['#4A4A4A','#A9A9A9','#2F2F2F']
        hatches = ['//','\\\\','||']
        width = 0.25
        x = np.arange(len(cross))
        for i, (col, color, hatch) in enumerate(zip(cross.columns, colors, hatches)):
            ax.bar(x + i*width, cross[col], width, color=color,
                   edgecolor='black', hatch=hatch, label=col.capitalize())
        ax.set_xticks(x + width)
        ax.set_xticklabels(cross.index)
        ax.set_xlabel('Class', fontsize=14, fontweight='bold')
        ax.set_ylabel('Documents', fontsize=14, fontweight='bold')
        ax.set_title('Sentiment Distribution by Class', fontsize=14, fontweight='bold')
        ax.tick_params(axis='both', labelsize=14)
```

```
        ax.tick_params(axis='both', labelsize=14)
        ax.legend()
        plt.tight_layout()
        plt.savefig(self.output_dir/'Sentiment_by_Class.png', dpi=300)
        plt.close()

    def create_multipanel_figure(self):
        import matplotlib.gridspec as gridspec
        import matplotlib.image as mpimg
        png_paths = {
            'class_comp': self.output_dir / 'Class_Comparison.png',
            'sent_topic': self.output_dir / 'Sentiment_by_Topic.png',
            'sent_class': self.output_dir / 'Sentiment_by_Class.png'
        }
        fig = plt.figure(figsize=(20, 16), dpi=300)
        gs = gridspec.GridSpec(2, 2, height_ratios=[2, 1], width_ratios=[1, 1])
        ax1 = fig.add_subplot(gs[0, 0])
        ax2 = fig.add_subplot(gs[0, 1])
        ax3 = fig.add_subplot(gs[1, :])
        for ax, key in zip([ax1, ax2, ax3], ['class_comp', 'sent_topic', 'sent_class']):
            img = mpimg.imread(png_paths[key])
            ax.imshow(img)
            ax.axis('off')
        labels = ['A', 'B', 'C']
        for ax, label in zip([ax1, ax2, ax3], labels):
            ax.text(-0.12, 1.08, label, transform=ax.transAxes,
                    fontsize=16, fontweight='bold', fontfamily='sans-serif',
                    va='top', ha='left')
        plt.tight_layout()
        fig.savefig(self.output_dir / 'Multipanel_Figure.png', dpi=300, bbox_inches='tight')
        fig.savefig(self.output_dir / 'Multipanel_Figure.pdf', format='pdf', bbox_inches='tight')
        plt.close(fig)

    def plot_umap_projection(self):
        umap_model = self.model.umap_model
        umap_emb = umap_model.transform(self.embeddings)
        df_plot = pd.DataFrame({
            'x': umap_emb[:, 0],
            'y': umap_emb[:, 1],
            'topic': self.model.topics_
        })
        df_plot = df_plot[df_plot.topic != -1]
        plt.figure(figsize=(16,10), dpi=300)
        sns.set_style("white")
        scatter = sns.scatterplot(data=df_plot, x='x', y='y', hue='topic',
                                  palette='tab20', s=10, alpha=0.4, legend=False)
        texts = []
        for tid in sorted(df_plot.topic.unique()):
            cx = df_plot[df_plot.topic == tid]['x'].mean()
            cy = df_plot[df_plot.topic == tid]['y'].mean()
            label = self.labels.get(tid, f"Topic {tid}")
            wrapped = "\n".join(label.split(" + ")[:2])
            texts.append(plt.text(cx, cy, wrapped, fontsize=12, fontweight='bold',
                                  bbox=dict(facecolor='white', alpha=0.7, edgecolor='none', pad=0.5)))
        adjust_text(texts, force_text=0.5, expand_points=(1.5,1.5),
                    arrowprops=dict(arrowstyle='->', color='black', lw=0.5, shrinkA=5, shrinkB=5))
        plt.title("UMAP Projection of Research Landscapes", fontsize=16, pad=20)
        plt.xlabel("UMAP Dimension 1")
        plt.ylabel("UMAP Dimension 2")
        sns.despine()
        plt.tight_layout()
        plt.savefig(self.output_dir/'UMAP_Projection.png', dpi=300, bbox_inches='tight')
        plt.close()

    def create_interactive_topic_map(self):
        fig = self.model.visualize_topics()
        fig.write_html(str(self.output_dir / 'Interactive_Topic_Map.html'))
        try:
            fig.write_image(str(self.output_dir / 'Interactive_Topic_Map.png'), scale=3)
        except Exception:
            pass

    def plot_temporal_evolution(self, top_n=15):
        df_y = self.df[self.df['publication_year'].notna()].copy()
        df_y = df_y[(df_y['publication_year'] >= 2021) & (df_y['publication_year'] <= 2026)]
        if df_y.empty:
            return
        top_topics = self.df['topic_id'].value_counts().head(top_n).index.tolist()
```

```
        rows = []
        for tid in top_topics:
            grp = df_y[df_y['topic_id'] == tid].groupby('publication_year').size().reset_index(name='count')
            grp['topic'] = tid
            grp['label'] = self.labels.get(tid, f"T{tid}")
            rows.append(grp)
        full = pd.concat(rows)
        dash_styles = ['solid','dash','dot','dashdot','longdash','longdashdot','5px 2px 2px 2px','3px 3px']
        colors = ['#1a1a1a','#333','#4A4A4A','#666','#808080','#A9A9A9','#C0C0C0','#D3D3D3']
        fig = px.line(full, x='publication_year', y='count', color='label', line_dash='label',
                      color_discrete_sequence=colors, line_dash_sequence=dash_styles)
        fig.update_layout(
            title='Topic Evolution Over Time',
            height=600,
            font_family='Arial',
            xaxis=dict(title='Publication Year', tickmode='linear', tick0=2021, dtick=1, range=[2021, 2026]),
            yaxis=dict(title='Number of Documents')
        )
        fig.write_html(str(self.output_dir/'Temporal_Evolution.html'))
        try:
            fig.write_image(str(self.output_dir/'Temporal_Evolution.png'), scale=3)
        except Exception:
            pass
        plt.close()

    def generate_all_visualizations(self):
        self.plot_class_comparison(top_n=15)
        self.plot_sentiment_by_topic(top_n=15)
        self.plot_sentiment_by_class()
        self.plot_temporal_evolution(top_n=15)
        self.plot_umap_projection()
        self.create_interactive_topic_map()
        try:
            if hasattr(self.model, '_hierarchical_topics') and self.model._hierarchical_topics is not None:
                fig_dendro = self.model.visualize_hierarchy(
                    hierarchical_topics=self.model._hierarchical_topics
                )
                fig_dendro.write_html(str(self.output_dir / 'Topic_Hierarchy_Dendrogram.html'))
        except Exception:
            pass
        try:
            heatmap_fig = self.model.visualize_heatmap(width=1200, height=1000)
            topic_ids = sorted([t for t in set(self.model.topics_) if t != -1])
            labels_full = [self.labels.get(tid, f"T{tid}") for tid in topic_ids]
            heatmap_fig.update_xaxes(tickvals=topic_ids, ticktext=labels_full, tickfont=dict(size=10), tickangle=45)
            heatmap_fig.update_yaxes(tickvals=topic_ids, ticktext=labels_full, tickfont=dict(size=10))
            heatmap_fig.update_layout(
                coloraxis_colorbar=dict(len=1.0, thickness=20, x=1.02, y=0.5,
                                        yanchor='middle', title='Similarity',
                                        title_font=dict(size=12, family='Arial, sans-serif')),
                margin=dict(l=300, r=200, t=80, b=200),
                width=1200, height=1000,
                font=dict(size=12, family="Arial, sans-serif")
            )
            heatmap_fig.write_html(str(self.output_dir / 'Topic_Similarity_Heatmap.html'))
        except Exception:
            pass


# ------------------------------------------------------------------------------
# EXPORT UTILITIES
# ------------------------------------------------------------------------------
class ExportManager:
    def __init__(self, df, model, labels, coherence_scores, cv_coherence, topic_diversity,
                 silhouette_highdim, silhouette_umap, output_dir=OUTPUT_DIR):
        self.df = df
        self.model = model
        self.labels = labels
        self.coherence_scores = coherence_scores
        self.cv_coherence = cv_coherence
        self.topic_diversity = topic_diversity
        self.silhouette_highdim = silhouette_highdim
        self.silhouette_umap = silhouette_umap
        self.output_dir = output_dir
        self.output_dir.mkdir(exist_ok=True)

    def export_main_results(self):
```

```
        export_df = self.df[['title','doi','topic_id','publication_year',
                             'technique','macro_theme','sentiment','sentiment_confidence']].copy()
        export_df['topic_label'] = export_df['topic_id'].map(self.labels)
        export_df.to_csv(self.output_dir/'Supplemental_Material_Topic_Model.csv', index=False)

    def export_topic_representatives(self, n_docs=10):
        representatives = []
        for topic_id in sorted(set(self.df['topic_id'])):
            topic_docs = self.df[self.df['topic_id'] == topic_id].nlargest(n_docs, 'sentiment_confidence')
            for idx, (_, row) in enumerate(topic_docs.iterrows(), 1):
                representatives.append({
                    'topic_id': topic_id,
                    'topic_label': self.labels.get(topic_id, ''),
                    'rank': idx,
                    'title': row['title'],
                    'doi': row['doi'],
                    'sentiment': row['sentiment']
                })
        pd.DataFrame(representatives).to_csv(self.output_dir/'Topic_Representative_Documents.csv', index=False)

    def export_coherence_scores(self):
        coh = [{'topic_id': tid, 'topic_label': self.labels.get(tid, ''),
                'coherence_score': score, 'document_count': (self.df['topic_id'] == tid).sum()}
               for tid, score in self.coherence_scores.items()]
        pd.DataFrame(coh).to_csv(self.output_dir/'Topic_Coherence_Scores.csv', index=False)

    def export_advanced_metrics(self):
        metrics = pd.DataFrame([{
            'C_v_coherence': self.cv_coherence,
            'Topic_diversity': self.topic_diversity,
            'Silhouette_high_dimension': self.silhouette_highdim,
            'Silhouette_from_umap': self.silhouette_umap
        }])
        metrics.to_csv(self.output_dir/'Advanced_Topic_Metrics.csv', index=False)

    def export_class_statistics(self):
        stats = []
        for tech in ['TEMPO', 'Control']:
            sub = self.df[self.df['technique'] == tech]
            if sub.empty:
                continue
            for theme in sub['macro_theme'].unique():
                subset = sub[sub['macro_theme'] == theme]
                stats.append({
                    'technique': tech,
                    'macro_theme': theme,
                    'document_count': len(subset),
                    'unique_topics': subset['topic_id'].nunique(),
                    'positive_pct': round((subset['sentiment'] == 'positive').sum() / len(subset) * 100, 2),
                    'neutral_pct': round((subset['sentiment'] == 'neutral').sum() / len(subset) * 100, 2),
                    'negative_pct': round((subset['sentiment'] == 'negative').sum() / len(subset) * 100, 2),
                    'year_range': f"{int(subset['publication_year'].min())}-{int(subset['publication_year'].max())}"
                })
        pd.DataFrame(stats).to_csv(self.output_dir/'Class_Statistics.csv', index=False)

    def export_all(self):
        self.export_main_results()
        self.export_topic_representatives()
        self.export_coherence_scores()
        self.export_advanced_metrics()
        self.export_class_statistics()

    def create_zip_archive(self):
        zip_path = Path('TEMPO_Review_Outputs.zip')
        with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zipf:
            for fp in self.output_dir.rglob('*'):
                if fp.is_file():
                    zipf.write(fp, fp.relative_to(self.output_dir.parent))
        return zip_path


# ------------------------------------------------------------------------
# MAIN EXECUTION
# ------------------------------------------------------------------------
def main():
    if IN_COLAB:
        ZOTERO_ID = userdata.get('ZOTERO_ID')
        ZOTERO_KEY = userdata.get('ZOTERO_KEY')
```

```
    else:
        ZOTERO_ID = input("Zotero ID: ")
        ZOTERO_KEY = input("Zotero Key: ")

    fetcher = ZoteroCollectionFetcher(ZOTERO_ID, ZOTERO_KEY)
    fetcher.build_collection_tree()
    items = fetcher.fetch_all_items()

    processor = ZoteroDataProcessor(items)
    df = processor.process_pipeline()

    # Benchmark models
    benchmark_models(df, model_names=['all-mpnet-base-v2', 'm3rg-iitd/matscibert'])

    topic_pipe = TopicModelingPipeline(df)
    model, labels = topic_pipe.train_pipeline(model_name='all-mpnet-base-v2', tune_umap=True)
    df['topic_id'] = model.topics_

    sent_pipe = SentimentAnalysisPipeline(df)
    sent_pipe.load_model()
    df = sent_pipe.analyze_sentiments()


# ============================================================================
# COMPUTATIONAL RIGOR SCORE FOR CNF LITERATURE (Applicable to associated research fields)
# ============================================================================

    # Definition of rigor indicators for CNF / biomaterials research

    rigor_keywords = {
        # ----- GOOD indicators (+1 each) -----
        'mechanical_testing': r'\b(mechanical\s+(test|property|strength|modulus)|tensile|stress-strain)\b',
        'rheological_char': r'\b(rheolog|viscosity|viscoelastic|oscillation|shear\s+rate)\b',
        'microscopy': r'\b(TEM|AFM|SEM|transmission\s+electron|atomic\s+force|scanning\s+electron)\b',
        'spectroscopy': r'\b(FTIR|XRD|XPS|NMR|Raman|spectroscop)\b',
        'chemical_quant': r'\b(carboxyl\s+content|conductometric\s+titration|zeta\s+potential|degree\s+of\s+oxidation)\b',
        'statistical_analysis': r'\b(statistical\s+(analysis|test)|ANOVA|t-test|p-value|standard\s+deviation|confidence\s+
        'replicates': r'\b(replicate|triplicate|duplicate|n\s*=\s*\d{2,})\b',
        'control_group': r'\b(control|comparison|reference|blank)\b',
        'longitudinal': r'\b(time\s+course|follow-up|over\s+time|longitudinal)\b',
        'validated_method': r'\b(validated|standard\s+protocol|ISO|ASTM)\b',

        # ----- POOR indicators (-1 each) -----
        'preliminary': r'\b(preliminary|exploratory|pilot|feasibility)\b',
        'limited_char': r'\b(limited\s+characterization|insufficient\s+data|lack\s+of\s+analysis)\b',
        'no_stats': r'\b(no\s+statistical\s+analysis|without\s+statistics|descriptive\s+only|qualitative\s+assessment)\b',
        'small_sample': r'\b(small\s+sample|n\s*=\s*[1-9]\b|few\s+samples)\b',
        'self_report': r'\b(self[- ]report|questionnaire\s+only|subjective)\b',
        'convenience': r'\b(convenience\s+sample|opportunistic|non-randomized)\b',
    }

    # Scoring function

    def compute_rigor_score(text: str) -> tuple:
        """
        Returns (score, details_dict) where score is raw integer sum.
        """
        if not isinstance(text, str):
            return 0, {}
        text_lower = text.lower()
        score = 0
        details = {}

        # Check GOOD indicators
        good_keys = ['mechanical_testing', 'rheological_char', 'microscopy', 'spectroscopy',
                     'chemical_quant', 'statistical_analysis', 'replicates', 'control_group',
                     'longitudinal', 'validated_method']
        for key in good_keys:
            if re.search(rigor_keywords[key], text_lower, re.IGNORECASE):
                score += 1
                details[key] = 'Present'
            else:
                details[key] = 'Absent'

        # Check POOR indicators
        poor_keys = ['preliminary', 'limited_char', 'no_stats', 'small_sample',
                     'self_report', 'convenience']
```

```
        for key in poor_keys:
            if re.search(rigor_keywords[key], text_lower, re.IGNORECASE):
                score -= 1
                details[key] = 'Present (penalty)'
            else:
                details[key] = 'Absent'

        return score, details

    # Apply to corpus (title + abstract)
    df['texto_completo'] = df['title'].fillna('') + " " + df['abstract'].fillna('')

    # Compute scores
    df['rigor_score'], df['rigor_details'] = zip(*df['texto_completo'].apply(compute_rigor_score))

    # Normalise to 0-100 scale
    min_score = df['rigor_score'].min()
    max_score = df['rigor_score'].max()
    if max_score > min_score:
        df['rigor_score_norm'] = (df['rigor_score'] - min_score) / (max_score - min_score) * 100
    else:
        df['rigor_score_norm'] = 50.0  # neutral if all equal

    # Export rigor scores to CSV
    rigor_csv_path = OUTPUT_DIR / 'SM_Rigor_Scores_CNF.csv'
    df[['title', 'doi', 'topic_id', 'technique', 'macro_theme',
        'rigor_score', 'rigor_score_norm', 'rigor_details']].to_csv(rigor_csv_path, index=False)


    # Visualizations

    sns.set_style("whitegrid")
    plt.rcParams.update({
        'font.family': 'sans-serif',
        'font.sans-serif': ['Arial', 'DejaVu Sans', 'Helvetica'],
        'font.size': 10,
        'pdf.fonttype': 42,
        'ps.fonttype': 42,
    })

    # Average rigor by topic (bar chart)
    topic_rigor = df.groupby('topic_id').agg(
        mean_rigor=('rigor_score_norm', 'mean'),
        count=('rigor_score_norm', 'count'),
        std_rigor=('rigor_score_norm', 'std')
    ).reset_index()

    # Exclude noise topic (labeled as "-1" in topics classification)
    topic_rigor = topic_rigor[topic_rigor['topic_id'] != -1]

    # Map topic labels
    def get_topic_label(tid):
        if tid in labels:
            return labels[tid]
        else:
            return f"Topic {tid}"

    topic_rigor['label'] = topic_rigor['topic_id'].apply(get_topic_label)

    # Sort by mean rigor
    topic_rigor_sorted = topic_rigor.sort_values('mean_rigor', ascending=True)

    fig, ax = plt.subplots(figsize=(12, 6))
    bars = ax.barh(topic_rigor_sorted['label'], topic_rigor_sorted['mean_rigor'],
                   xerr=topic_rigor_sorted['std_rigor'], capsize=5,
                   color='darkblue', alpha=0.7, edgecolor='black')
    ax.set_xlabel('Computational Rigor Score (0-100)', fontsize=12)
    ax.set_title('Methodological Rigor by Thematic Cluster (CNF Literature)', fontsize=14, fontweight='bold')
    ax.axvline(x=50, color='red', linestyle='--', linewidth=1.5, label='Moderate Rigor Threshold')
    ax.legend()
    plt.tight_layout()
    rigor_topic_path = OUTPUT_DIR / 'SM_Figure_Rigor_by_Topic_CNF.png'
    plt.savefig(rigor_topic_path, dpi=300, bbox_inches='tight')
    plt.close(fig)

    # Rigor distribution by class (TEMPO vs Control)
    fig, ax = plt.subplots(figsize=(8, 6))
    sns.boxplot(data=df[df['topic_id'] != -1], x='technique', y='rigor_score_norm',
```

```
    sns.boxplot(data=df[df['topic_id'] != -1], x='technique', y='rigor_score_norm',
                palette={'TEMPO': '#4AAAAA', 'Control': '#A9A9A9'}, ax=ax)
    sns.swarmplot(data=df[df['topic_id'] != -1], x='technique', y='rigor_score_norm',
                  color='black', alpha=0.4, size=3, ax=ax)
    ax.set_xlabel('Class', fontsize=12)
    ax.set_ylabel('Rigor Score (0–100)', fontsize=12)
    ax.set_title('Rigor Score Distribution by Class', fontsize=14, fontweight='bold')
    plt.tight_layout()
    rigor_class_path = OUTPUT_DIR / 'SM_Figure_Rigor_by_Class_CNF.png'
    plt.savefig(rigor_class_path, dpi=300, bbox_inches='tight')
    plt.close(fig)

    # Heatmap of rigor indicators – count per topic
    indicator_cols = [k for k in rigor_keywords.keys() if k not in ['preliminary','limited_char','no_stats','small_sample']

    for key in indicator_cols:
        df[f'flag_{key}'] = df['texto_completo'].apply(
            lambda t: 1 if re.search(rigor_keywords[key], str(t), re.IGNORECASE) else 0
        )

    # Aggregate per topic
    topic_indicator = df[df['topic_id'] != -1].groupby('topic_id')[[f'flag_{k}' for k in indicator_cols]].mean()
    topic_indicator.index = topic_indicator.index.map(get_topic_label)

    # Plot heatmap
    fig, ax = plt.subplots(figsize=(14, max(6, 0.4 * len(topic_indicator))))
    sns.heatmap(topic_indicator, annot=True, fmt='.2f', cmap='Blues', cbar_kws={'label': 'Proportion of documents'},
                linewidths=0.5, ax=ax)
    ax.set_title('Prevalence of Rigor Indicators by Topic', fontsize=14, fontweight='bold')
    ax.set_xlabel('Indicator')
    ax.set_ylabel('Topic')
    plt.tight_layout()
    rigor_heatmap_path = OUTPUT_DIR / 'SM_Figure_Rigor_Heatmap_CNF.png'
    plt.savefig(rigor_heatmap_path, dpi=300, bbox_inches='tight')
    plt.close(fig)

    print("\n=== Computational Rigor Scores (CNF) ===")
    print(f"Mean rigor score (all docs): {df['rigor_score_norm'].mean():.2f} ± {df['rigor_score_norm'].std():.2f}")
    print(f"Min: {df['rigor_score_norm'].min():.2f}, Max: {df['rigor_score_norm'].max():.2f}")
    print("\nAverage rigor by topic (top 5):")
    print(topic_rigor_sorted[['label', 'mean_rigor', 'count']].tail(5).to_string(index=False))


    viz = PublicationQualityVisualizations(df, model, labels, embeddings=topic_pipe.embeddings)
    viz.generate_all_visualizations()
    viz.create_multipanel_figure()

    exp = ExportManager(df, model, labels, topic_pipe.coherence_scores,
                        topic_pipe.cv_coherence, topic_pipe.topic_diversity,
                        topic_pipe.silhouette_highdim, topic_pipe.silhouette_umap)
    exp.export_all()
    zip_path = exp.create_zip_archive()

    if IN_COLAB:
        from google.colab import files
        files.download(str(zip_path))

if __name__ == '__main__':
    main()
```

```
--- Benchmarking model: all-mpnet-base-v2 ---
modules.json: 100%                                                  349/349 [00:00<00:00, 33.9kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
config_sentence_transformers.json: 100%                             116/116 [00:00<00:00, 5.71kB/s]
README.md: 100%                                                     11.6k/11.6k [00:00<00:00, 1.18MB/s]
sentence_bert_config.json: 100%                                     53.0/53.0 [00:00<00:00, 4.43kB/s]
config.json: 100%                                                   571/571 [00:00<00:00, 51.4kB/s]
model.safetensors: 100%                                             438M/438M [00:05<00:00, 146MB/s]
Loading weights: 100%                                               199/199 [00:00<00:00, 3632.08it/s]
tokenizer_config.json: 100%                                         363/363 [00:00<00:00, 31.1kB/s]
vocab.txt: 100%                                                     232k/232k [00:00<00:00, 9.82MB/s]
tokenizer.json: 100%                                                466k/466k [00:00<00:00, 25.2MB/s]
special_tokens_map.json: 100%                                       239/239 [00:00<00:00, 22.9kB/s]
config.json: 100%                                                   190/190 [00:00<00:00, 10.4kB/s]
100%|██████████| 18/18 [00:00<00:00, 266.04it/s]

--- Benchmarking model: m3rg-iitd/matscibert ---
config.json: 100%                                                   620/620 [00:00<00:00, 69.9kB/s]
tokenizer_config.json: 100%                                         323/323 [00:00<00:00, 34.6kB/s]
vocab.txt: 100%                                                     228k/228k [00:00<00:00, 10.1MB/s]
tokenizer.json: 100%                                                467k/467k [00:00<00:00, 28.0MB/s]
special_tokens_map.json: 100%                                       112/112 [00:00<00:00, 12.2kB/s]
model.safetensors: 100%                                             440M/440M [00:05<00:00, 89.0MB/s]
Loading weights: 100%                                               197/197 [00:00<00:00, 3858.05it/s]
[transformers] BertModel LOAD REPORT from: m3rg-iitd/matscibert
Key                                         | Status     |
--------------------------------------------+------------+-
cls.predictions.transform.LayerNorm.bias    | UNEXPECTED |
cls.predictions.bias                        | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight  | UNEXPECTED |
cls.predictions.transform.dense.weight      | UNEXPECTED |
cls.predictions.transform.dense.bias        | UNEXPECTED |
pooler.dense.bias                           | MISSING    |
pooler.dense.weight                         | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING:  those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
100%|██████████| 18/18 [00:00<00:00, 282.07it/s]
                  model  C_v_coherence  topic_diversity  silhouette_highdim  \
0     all-mpnet-base-v2       0.646393         0.847368            0.071887
1  m3rg-iitd/matscibert       0.554620         0.763158            0.021504

   silhouette_umap  num_topics  noise_ratio
0         0.184347          19     0.169130
1         0.027750          19     0.293924
```

END